# Personal Information Management

A local command-line PIM for notes, tasks, events and contacts

**COMP3211 · Group 89**

Review candidate · 23 October 2026

# Creating a PIR

**FR-03 / Task fields**

A task needs a description and a local deadline.

**COMMAND**

```
add task "draft" "2026-10-01 09:00"
```

**OBSERVED CLI OUTPUT (AFTER CREATING NOTE 1)**

```
Created task 2
id: 2
type: task
description: draft
deadline: 2026-10-01 09:00
```

# Defining a search criterion

**FR-26 / One search command defines and runs the criterion**

```
search (type = "task" && deadline < "2026-10-02 00:00")
       || text contains "memo"
```

Parentheses group the task and deadline test.
&& requires both conditions; || accepts either branch.

Quoted literals are required. The parser builds criterion objects; it does not run Python input.

# Searching for PIRs

### FR-27 / Evaluate against the current collection

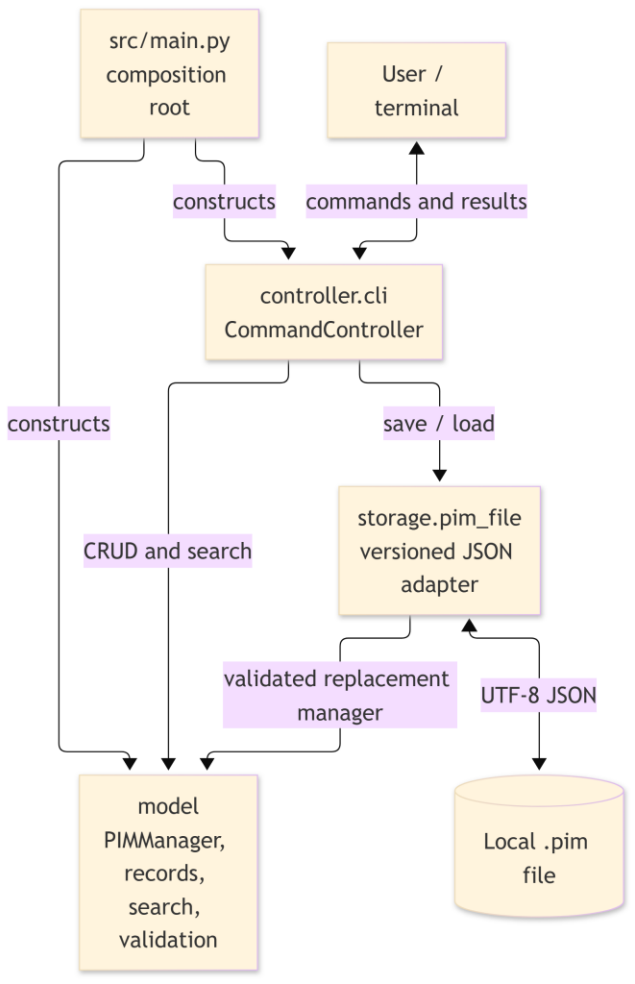
The compound criterion matches two of the four records:

```
1  |  note  |  memo
2  |  task  |  draft
```

### FR-39 / Results appear in increasing ID order

After updating task 2 and deleting note 1, a saved snapshot restores three records. The next new note receives ID 5.

# Layered design and component relationships




**CommandController: terminal view + control**

Calls the model for CRUD/search and storage for files.

**PIMManager owns immutable record objects**

Note, Task, Event and Contact. Search delegates to Criterion.matches(record).

**storage.pim_file: persistence adapter**

Saves versioned JSON; loads a validated replacement manager.

MVC-style separation keeps model tests independent of terminal and file I/O.

# Lesson from unit testing

FINDING

Deep valid search expressions exceeded Python's recursion limit.

CHANGE

Regression tests led to iterative parsing and evaluation.

VERIFIED 23 OCTOBER 2026

**51 tests passed · 416/421 model source lines · 98.81%**

Python 3.11.9 / 3.12.3 / 3.13.5; source-line trace excludes 0/None.